

02 Cache 开卷速查

一分钟速查

Cache 题几乎一定是计算题：容量、地址拆分、hit/miss、byte returned、miss rate、LRU/write-back、优化解释。复习课明确说 generic cache organization 那张图 (review.pdf p17) “肯定会考”。

来源优先级：

- QUIZ/ics2-quiz.pdf：Problem 1，正式 quiz 风格的 Cache 综合题。
- QUIZ/ics2-quiz-1.pdf：Problem 1 答案/讲解版。
- EXE/EXE3.pdf：Problem 1，基础 cache 表 hit/miss；Problem 2 是 storage hierarchy 概念。
- EXE/EXE3-sol.pdf：对应答案。
- EXE/EXE4.pdf：Problem 1、Problem 2，更完整的 miss rate、dirty/write-back、middle-bit indexing、硬件参数优化。
- EXE/EXE4-sol.pdf：对应答案。

注意：Cache 和 VM 都会用到“地址切分”思想。Cache 是 tag / set index / block offset；VM 是 PPN/PPO 或 VPN/VPO。VIPT 题会把这两套切分联系起来。

题型对应来源：

题型	主要来源
cache 总容量 / tag、set index、block offset 位数	QUIZ/ics2-quiz.pdf Problem 1; EXE/EXE3.pdf Problem 1; EXE/EXE4.pdf Problem 2
给 cache 表判断 hit/miss 和 byte returned	QUIZ/ics2-quiz.pdf Problem 1; EXE/EXE3.pdf Problem 1
LRU、write-back、dirty、flush 写回次数	EXE/EXE4.pdf Problem 2
程序访存 miss rate	QUIZ/ics2-quiz.pdf Problem 1; EXE/EXE4.pdf Problem 1/2

题型	主要来源
middle-bit indexing / 硬件参数优化 / 软件地址调整	QUIZ/ics2-quiz.pdf Problem 1; EXE/EXE4.pdf Problem 2

核心句：

Cache 题不要凭感觉判断命中；每次都先拆地址，再看 valid，再比 tag，最后用 offset 取 byte。

基本概念

- Locality：局部性，是 cache 有效的根本原因。
- Temporal locality：时间局部性，最近访问过的数据可能再次访问。
- Spatial locality：空间局部性，访问某地址后可能访问附近地址。
- Set：cache 的组。
- Line / block：cache 中保存的一块连续内存。
- Valid bit：有效位，valid=0 时 tag 对上也 miss。
- Dirty bit：脏位，write-back 下表示 cache line 被改过但还没写回内存。

基本参数：

参数	含义
$S = 2^s$	set 数
E	每个 set 的 line 数，也叫 associativity
$B = 2^b$	block size，单位 bytes
m	物理地址位数
$C = S * E * B$	cache data capacity，不含 tag/valid/dirty 开销

地址拆分：

| tag: t bits | set index: s bits | block offset: b bits |

必背公式：

$b = \log_2(B)$
 $s = \log_2(S)$
 $t = m - s - b$
 $C = S * E * B$

快速拆地址：

```

block_number = address / B
set_index    = block_number % S
tag          = block_number / S
offset       = address % B

```

小技巧：如果 $s + b$ 不是 4 的整数倍，不能只靠十六进制位数直接截断得到 tag。把地址当作无符号位串做逻辑右移：右移 b 位等价于除以 B 得到 `block_number`，再右移 s 位等价于除以 s 得到 `tag`。每右移 1 位就是 $/2$ ，考试时可以用这个快速算。比如 $s+b=6$ 时把 HEX 最右位抛掉（4 位 DEC）后剩余的部分 DEC 右移 2 位，即 $/4$

题型模板

1. 容量与地址位宽

来源：QUIZ/ics2-quiz.pdf Problem 1；EXE/EXE3.pdf Problem 1；EXE/EXE4.pdf Problem 2。

例：16-bit machine, 2-way, 2 sets, block size 8 bytes。

```

C = S * E * B = 2 * 2 * 8 = 32 bytes
b = log2(8) = 3
s = log2(2) = 1
t = 16 - 1 - 3 = 12

```

答案：

tag 12 bits, set-index 1 bit, block-offset 3 bits

开卷提示：题目问 cache size 时，一般只算 data capacity，不算 valid/tag/dirty。

2. 给地址判断 hit/miss/byte returned

来源：QUIZ/ics2-quiz.pdf Problem 1；EXE/EXE3.pdf Problem 1。

步骤：

1. 用 B 算 offset。
2. 用 s 算 set index。
3. 剩余高位是 tag。
4. 到对应 set 内找 `valid=1` && tag match 的 line。
5. 命中时返回 `Byte[offset]`。
6. miss 时 byte returned 通常写 - 或 --，并按题意更新 cache。

易错点：

- tag 对上但 valid=0，仍然 miss。
- 前面的访问可能已经按 LRU evict 了某 line。
- 表里的 tag 不是完整地址，不要拿完整地址直接比。

3. LRU / write-back / dirty

来源：EXE/EXE4.pdf Problem 2。

LRU 更新：

- hit：该 line 变成最近使用。
- miss 且有 invalid line：优先填 invalid line。
- miss 且 set 满：evict 最久未使用 line。

Write-back：

- write hit：改 cache line，dirty=1。
- write miss：通常装入 block 后写，dirty=1。
- dirty line 只有 eviction 或最后 flush 时写回内存。

写回次数模板：

总写回次数 = 过程中 dirty eviction 次数 + 最后 flush 时仍 dirty 的 line 数

4. 程序 miss rate

来源：QUIZ/ics2-quiz.pdf Problem 1；EXE/EXE4.pdf Problem 1/2。

流程：

1. 明确只考虑 data cache，忽略 instruction cache。
2. 按代码顺序列出每次 data access 的地址。
3. 每个地址拆成 set/tag/offset。
4. 逐步更新 cache 状态。
5. 统计 misses / total accesses。

常见数组题：

$C[i] = A[i] * B[i] + C[i];$

至少涉及 $A[i]$ 、 $B[i]$ 、 $C[i]$ 的读，是否把写 $C[i]$ 单独计入要看题目说明。

5. middle-bit indexing

来源：EXE/EXE4.pdf Problem 2。

标准答法：

如果用高位做 set index，顺序扫描数组时高位长时间不变，大量连续 blocks 会落到同一个 set，其他 sets 空着。用中间位做 set index，可以让相邻 blocks 分散到不同 sets，更好利用整个 cache。

关键词：high-order bits、middle bits、sequential scan、conflict miss。

6. 硬件/软件优化 miss rate

来源：QUIZ/ics2-quiz.pdf Problem 1；EXE/EXE4.pdf Problem 1/2。

硬件方向：

- 增大 block size：顺序访问时利用 spatial locality，但总容量不变时 line 数减少，可能加重冲突。
- 增大 associativity / lines per set：通常减少 conflict misses。
- 增大 sets：效果依赖地址映射，不能无脑说一定更好。

软件方向：

- 调整数组起始地址，避免多个数组映射到同一 set。
- 改循环顺序，让内层访问连续地址。
- blocking / tiling，小块访问，提高局部性。

QUIZ 典型答法：把某个数组起始地址移动一点，例如让 $c[0]$ 从 $0xc00$ 改到 $0xc08$ ，避免 A/B/C 全部冲突。

考场检查清单

1. 先写 S/E/B/m。
2. 算 s/b/t。
3. 每个地址拆 tag/set/offset。
4. hit 必须 valid=1 且 tag 相同。
5. miss 后按 LRU 更新状态。
6. write-back 题要跟 dirty bit。
7. miss rate 题先列访问序列，再数 hit/miss。
8. 优化题先判断是 spatial locality、conflict miss，还是 capacity 问题。

高频术语

- Cache hit / miss: 缓存命中 / 不命中。
- Cache line / block: 缓存行 / 块。
- Set associative cache: 组相联缓存。
- Direct-mapped cache: 直接映射缓存, $E = 1$ 。
- Valid bit: 有效位。
- Dirty bit: 脏位。
- Tag / set index / block offset: 标记 / 组索引 / 块内偏移。
- LRU: Least Recently Used, 最近最少使用。
- Write-back: 写回。
- Conflict miss: 冲突不命中。
- Cold miss / compulsory miss: 冷不命中 / 强制不命中。